



Universidad Autónoma De Chiapas
Escuela de tecnologías digitales aplicadas
Licenciatura en Sistemas Computacionales



Proyecto WikiUnach - Biblioteca Digital

Asignatura:

Lenguajes de Consulta

Docente:

Dr. Luis Gutiérrez Alfaro

Alumnos:

Axel Ramos Monroy

Carlos Adolfo Ortiz Ortiz

Ruth Carolina Moscoso Villatoro

Luis Roberto Miranda De La Cruz

4to. Semestre Grupo "K"

Índice

INTRODUCCION	4
OBJETIVOS.....	5
<i>Objetivo General.....</i>	<i>5</i>
<i>Objetivos específicos.....</i>	<i>5</i>
JUSTIFICACION.....	6
ALCANCE.....	6
CRONOGRAMA.....	7
REQUERIMIENTOS	8
<i>Requerimientos Funcionales (RF).....</i>	<i>8</i>
<i>Requerimientos No Funcionales (RNF).....</i>	<i>9</i>
DISEÑO DE BASE DE DATOS.....	10
<i>Esquema ER.....</i>	<i>10</i>
.....	10
<i>Normalización.....</i>	<i>11</i>
Primera Forma Normal (1NF)	11
Segunda Forma Normal (2NF).....	12
Cómo lo cumple WikiUnach	13
<i>Script SQL.....</i>	<i>14</i>

<i>Vistas (Implementación Programática).....</i>	<i>17</i>
Vista Unificada de Archivos Adjuntos (vw_ArchivosCompletos)	19
<i>Consultas simples, agregadas y con múltiples tablas.....</i>	<i>21</i>
<i>IMPLEMENTACION AWS.....</i>	<i>22</i>
<i>Infraestructura.....</i>	<i>22</i>
<i>Conexión remota.....</i>	<i>23</i>
<i>Seguridad.....</i>	<i>23</i>
<i>Respaldo.....</i>	<i>25</i>
<i>Costo estimado.....</i>	<i>26</i>
<i>APLICACIÓN DE CLIENTE.....</i>	<i>27</i>
<i>Interfaz visual.....</i>	<i>27</i>
<i>CRUD (Create, Read, Update, Delete).....</i>	<i>28</i>
<i>Conexión con AWS.....</i>	<i>29</i>
<i>Ejemplos de manejo de errores.....</i>	<i>29</i>
<i>CONCLUSION.....</i>	<i>31</i>

INTRODUCCION

Uno de los problemas más comunes dentro de la vida universitaria es la pérdida y dispersión del material académico generado por los estudiantes. Los apuntes, guías y recursos suelen circular por medios informales y terminan desapareciendo. WikiUnach nace para atender esta necesidad, ofreciendo a la comunidad de la UNACH una plataforma de escritorio (C# WinForms) robusta y conectada a la nube (AWS), donde el conocimiento puede organizarse, conservarse y estar disponible, vinculado directamente a las facultades y materias correspondientes.

OBJETIVOS

Objetivo General

Desarrollar un entorno colaborativo digital mediante una arquitectura cliente-servidor, que permita el almacenamiento, indexación, catalogación y recuperación eficiente del material académico de la UNACH, utilizando servicios en la nube (AWS RDS y AWS S3) los cuales permitan una aplicación con una interfaz visual sencilla y cómoda para los usuarios, con un background que permita el acceso 24/7 a la información requerida.

Objetivos específicos

1. Diseñar e implementar una base de datos relacional normalizada en MySQL alojada en AWS RDS.
2. Desarrollar un sistema de gestión de archivos binarios utilizando AWS S3 con control de acceso a través de URLs pre-firmadas.
3. Construir una aplicación cliente en C# (Visual Studio 2019) que consuma estos servicios, implementando un CRUD completo y un sistema de control de versiones.
4. Integrar validaciones y manejo de errores, así como una librería de bajo nivel (híbrido C++/Ensamblador) para autorizar el acceso de administrativos.

JUSTIFICACION

La fragmentación del conocimiento afecta el rendimiento y la colaboración estudiantil. Una arquitectura tradicional local sería insuficiente para el volumen de datos (archivos pesados multimedia) que maneja una universidad. Al optar por una arquitectura basada en AWS RDS y S3, el proyecto garantiza escalabilidad automática, alta disponibilidad y un costo de almacenamiento eficiente, delegando la persistencia física a la nube y resolviendo el problema de accesibilidad permanente para los estudiantes.

ALCANCE

El sistema abarca la gestión de perfiles de usuario, autenticación cifrada (SHA-256), administración de materias por facultades, carga/descarga de tareas, control de revisiones colaborativas, un motor de comentarios, un sistema de votación (Like/Dislike) y un panel de administración restringido (Activado por secuencias de teclado y DLL nativa).

CRONOGRAMA

Fase de Desarrollo	Actividades Realizadas	Duración
1. Análisis y Diseño	Levantamiento de requerimientos, diseño del modelo Entidad-Relación y normalización de tablas hasta la 3NF.	12 hrs
2. Configuración Cloud	Despliegue de instancia AWS RDS (MySQL), configuración de Security Groups (puerto 3306), creación de bucket S3 y políticas IAM.	10 hrs
3. Desarrollo Base de Datos	Creación de scripts SQL, declaración de llaves foráneas, constraints de cascada e índices UNIQUE.	8 hrs
4. Desarrollo Cliente C#	Creación de la interfaz gráfica (WinForms), diseño adaptable mediante propiedades Anchor/Dock, y programación del CRUD con MySqlConnection.	16 hrs
5. Programación de Sistemas	Desarrollo, compilación e integración (P/Invoke) de la librería nativa VerificadorAdmin.dll en C++ y Ensamblador x86.	8 hrs
6. Pruebas y Depuración	Pruebas de carga/descarga en AWS S3, validación de integridad referencial y elaboración de documentación final.	10 hrs
TOTAL		64 Horas

REQUERIMIENTOS

Requerimientos Funcionales (RF)

RF01 - Gestión de Sesión: El sistema debe permitir la creación de usuarios identificados por un correo (o matrícula) únicos, así como autenticar usuarios evaluando su correo y contraseña, los cuales son almacenados aplicando hash SHA-256.

RF02 - Niveles de Acceso: El sistema manejará roles de usuario (Visitante, Estudiante, Profesor, Administrador). Los diferentes tipos de usuarios tendrán permisos exclusivos de cada rol, lo que permitirá una gestión jerárquica entre usuarios.

RF03 - CRUD de Contenido: El sistema debe permitir crear, leer, actualizar y eliminar páginas Wiki, verificando la integridad entre la información de los dos servicios de AWS, así como registrar un historial inmutable de las modificaciones (Commits).

RF04 - Gestión de Archivos: La plataforma debe permitir adjuntar archivos a las publicaciones, subiéndolos a AWS S3 y recuperándolos mediante URLs pre-firmadas válidas por 15 minutos.

RF05 - Administración de Usuarios: Existirá un panel de administración (FrmAdminUsuarios) que permita operaciones CRUD sobre las cuentas de usuario, así como las diferentes tareas publicadas en la aplicación.

RF06 - Interacción Social: Los usuarios autenticados podrán votar positivamente (Like) o negativamente (Dislike) las diferentes tareas, además podrán realizar comentarios en las publicaciones.

Requerimientos No Funcionales (RNF)

RNF01 - Arquitectura de Software: La aplicación cliente estará desarrollada en C# WinForms bajo el framework .NET dentro del IDE Visual Studio 2019.

RNF02 - Integridad de Transacciones: Las operaciones de actualización complejas (ej. reemplazar archivos y guardar historial) deben ejecutarse bajo transacciones atómicas (MySQLTransaction) con capacidad de realizar un Rollback dentro del periodo de 15 minutos.

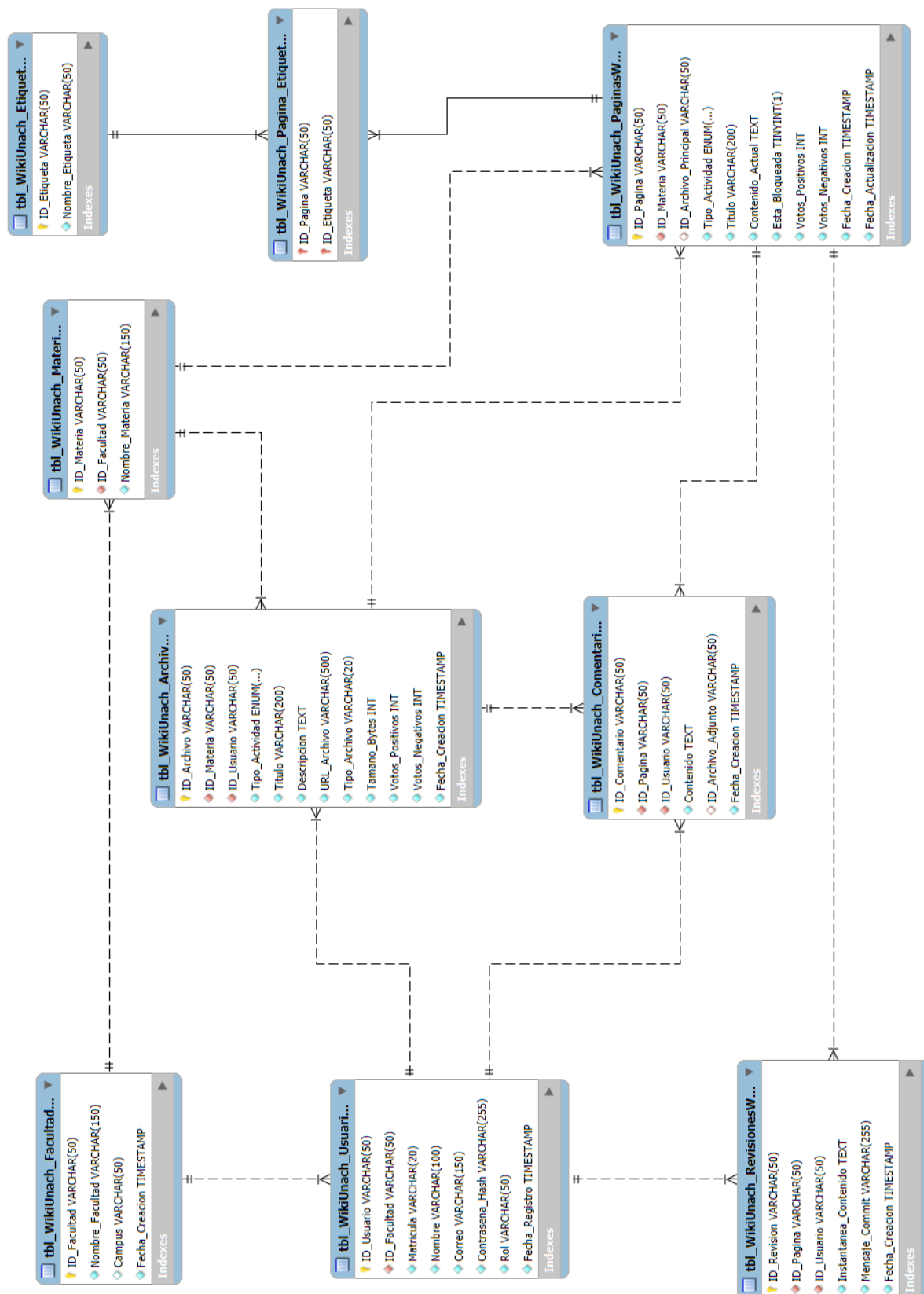
RNF03 - Seguridad a Bajo Nivel: El acceso al modo administrador debe ser validado por una biblioteca de vínculos dinámicos externa (VerificadorAdmin.dll) escrita en C++ y Ensamblador x86.

RNF04 - Compatibilidad de Plataforma: Debido al uso de ensamblador en línea (__asm), todo el proyecto (C# y C++) debe estar compilado estrictamente en arquitectura Win32 (x86).

RNF05 - Conectividad de Red: La base de datos RDS debe soportar conexiones entrantes TCP a través del puerto 3306 desde el cliente local.

DISEÑO DE BASE DE DATOS

Esquema ER



Normalización

Primera Forma Normal (1NF)

Regla: Todos los atributos deben ser atómicos (indivisibles), no deben existir grupos repetitivos ni arreglos dentro de una columna, y cada tabla debe tener una llave primaria que identifique de forma única cada fila.

Cómo lo cumple WikiUnach:

Atomicidad: En lugar de tener un campo llamado Nombre_Completo_Y_Matricula, la tabla tbl_WikiUnach_Usuarios divide la información en atributos atómicos y específicos: Matricula, Nombre, Correo.

Eliminación de grupos repetitivos: Un error común en bases de datos no normalizadas sería guardar las etiquetas de una página Wiki separadas por comas en una sola columna (Ej. Etiquetas: "Programación, C#, SQL"). En tu diseño, esto se solucionó creando la tabla catálogo tbl_WikiUnach_Etiquetas y la tabla intermedia tbl_WikiUnach_Pagina_Etiquetas.

Llave Primaria Única: Absolutamente todas las tablas tienen un identificador único, como ID_Usuario, ID_Archivo o ID_Pagina, utilizando cadenas VARCHAR(50) para almacenar GUIDs.

Segunda Forma Normal (2NF)

Regla: La base de datos debe estar en 1NF. Además, todos los atributos que no son clave deben depender funcionalmente de la totalidad de la llave primaria. (Esta regla aplica principalmente para tablas con llaves primarias compuestas).

Cómo lo cumple WikiUnach:

La gran mayoría de tus tablas (Materias, Archivos, Usuarios) utilizan una llave primaria simple (de una sola columna). Por regla matemática, cualquier tabla en 1NF con una llave primaria simple ya está automáticamente en 2NF, porque no hay "partes" de la llave de las cuales depender.

Llave compuesta: La tabla `tbl_WikiUnach_Pagina_Etiquetas` tiene una llave primaria compuesta por (`ID_Pagina`, `ID_Etiqueta`). Como esta tabla solo sirve para vincular y no tiene columnas adicionales de datos (como "`Fecha_Asignacion`"), no existe riesgo de dependencias parciales. El diseño es perfecto para 2NF.

Tercera Forma Normal (3NF)

Regla: La base de datos debe estar en 2NF. Además, no deben existir dependencias transitivas. Esto significa que ningún atributo no-clave debe depender de otro atributo no-clave; toda la información debe depender directa y exclusivamente de la Llave Primaria.

Cómo lo cumple WikiUnach

Catálogo de Facultades y Licenciaturas: Si la tabla `tbl_WikiUnach_Usuarios` incluyera las columnas `ID_Facultad` y `Nombre_Facultad`, habría una dependencia transitiva (El Nombre de la Facultad dependería del ID de la Facultad, no del Usuario). Para evitar esto, tu diseño almacena únicamente el `ID_Facultad` y el `ID_Licenciatura` en la tabla de usuarios como Llaves Foráneas (FOREIGN KEY). La información descriptiva vive exclusivamente en `tbl_WikiUnach_Facultades` y `tbl_WikiUnach_Licenciaturas`.

Integridad de Archivos y Tareas: En la tabla `tbl_WikiUnach_Archivos`, datos como el `Titulo`, `URL_Archivo` y `Tamano_Bytes` dependen única y exclusivamente del `ID_Archivo`. Si quisiéramos saber el semestre de la materia a la que pertenece ese archivo, no lo guardamos repetido en la tabla de archivos; hacemos un JOIN con la tabla `tbl_WikiUnach_Materias` a través del `ID_Materia`.

Script SQL

```
-- MySQL Workbench Forward Engineering
```

```
SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE,
SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_
FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';
```

```
CREATE SCHEMA IF NOT EXISTS `WikiUnach` DEFAULT CHARACTER SET utf8mb4 COLLATE
utf8mb4_0900_ai_ci ;
USE `WikiUnach` ;
```

```
-- Table `tbl_WikiUnach_Facultades`
```

```
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_Facultades` (
  `ID_Facultad` VARCHAR(50) NOT NULL,
  `Nombre_Facultad` VARCHAR(150) NOT NULL,
  `Campus` VARCHAR(50) NULL DEFAULT NULL,
  `Fecha_Creacion` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`ID_Facultad`))
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;
```

```
-- Table `tbl_WikiUnach_Licenciaturas`
```

```
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_Licenciaturas` (
  `ID_Licenciatura` VARCHAR(50) NOT NULL,
  `ID_Facultad` VARCHAR(50) NOT NULL,
  `Nombre_Licenciatura` VARCHAR(150) NOT NULL,
  PRIMARY KEY (`ID_Licenciatura`),
  INDEX `fk_Licenciatura_Facultad` (`ID_Facultad` ASC) VISIBLE,
  CONSTRAINT `fk_Licenciatura_Facultad` FOREIGN KEY (`ID_Facultad`)
    REFERENCES `WikiUnach`.`tbl_WikiUnach_Facultades` (`ID_Facultad`) ON DELETE CASCADE)
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;
```

```
-- Table `tbl_WikiUnach_Materias`
```

```
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_Materias` (
  `ID_Materia` VARCHAR(50) NOT NULL,
  `Nombre_Materia` VARCHAR(150) NOT NULL,
  `ID_Licenciatura` VARCHAR(50) NOT NULL,
  `Semestre` VARCHAR(10) NOT NULL,
  PRIMARY KEY (`ID_Materia`),
  INDEX `fk_Materia_Licenciatura` (`ID_Licenciatura` ASC) VISIBLE,
  CONSTRAINT `fk_Materia_Licenciatura` FOREIGN KEY (`ID_Licenciatura`)
    REFERENCES `WikiUnach`.`tbl_WikiUnach_Licenciaturas` (`ID_Licenciatura`) ON DELETE CASCADE)
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;
```

```
-- Table `tbl_WikiUnach_Usuarios`
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_Usuarios` (
  `ID_Usuario` VARCHAR(50) NOT NULL,
  `ID_Facultad` VARCHAR(50) NOT NULL,
  `ID_Licenciatura` VARCHAR(50) NULL DEFAULT NULL,
  `Semestre` VARCHAR(10) NULL DEFAULT NULL,
  `Matricula` VARCHAR(20) NULL DEFAULT NULL,
  `Nombre` VARCHAR(100) NOT NULL,
  `Correo` VARCHAR(150) NOT NULL,
  `Contrasena_Hash` VARCHAR(255) NOT NULL,
  `Rol` VARCHAR(50) NOT NULL,
  `Fecha_Registro` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`ID_Usuario`),
  UNIQUE INDEX `Correo_UNIQUE` (`Correo` ASC) VISIBLE,
  UNIQUE INDEX `Matricula_UNIQUE` (`Matricula` ASC) VISIBLE,
  CONSTRAINT `fk_Usuario_Facultad` FOREIGN KEY (`ID_Facultad`)
    REFERENCES `WikiUnach`.`tbl_WikiUnach_Facultades` (`ID_Facultad`) ON DELETE CASCADE,
  CONSTRAINT `fk_Usuario_Licenciatura` FOREIGN KEY (`ID_Licenciatura`)
    REFERENCES `WikiUnach`.`tbl_WikiUnach_Licenciaturas` (`ID_Licenciatura`) ON DELETE SET NULL ON
  UPDATE CASCADE)
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;
```

```
-- Table `tbl_WikiUnach_Archivos`
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_Archivos` (
  `ID_Archivo` VARCHAR(50) NOT NULL,
  `ID_Materia` VARCHAR(50) NOT NULL,
  `ID_Usuario` VARCHAR(50) NOT NULL,
  `Tipo_Actividad` ENUM('Tarea', 'Proyecto', 'Ensayo', 'Investigación', 'Diagrama', 'Otro') NOT NULL,
  `Titulo` VARCHAR(200) NOT NULL,
  `Descripcion` TEXT NOT NULL,
  `URL_Archivo` VARCHAR(500) NOT NULL,
  `Tipo_Archivo` VARCHAR(20) NOT NULL,
  `Tamano_Bytes` INT NOT NULL,
  `Votos_Positivos` INT NOT NULL DEFAULT '0',
  `Votos_Negativos` INT NOT NULL DEFAULT '0',
  `Fecha_Creacion` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`ID_Archivo`),
  CONSTRAINT `fk_Archivo_Materia` FOREIGN KEY (`ID_Materia`)
    REFERENCES `WikiUnach`.`tbl_WikiUnach_Materias` (`ID_Materia`) ON DELETE CASCADE,
  CONSTRAINT `fk_Archivo_Usuario` FOREIGN KEY (`ID_Usuario`)
    REFERENCES `WikiUnach`.`tbl_WikiUnach_Usuarios` (`ID_Usuario`))
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;
```

```
-- Table `tbl_WikiUnach_PaginasWiki`
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_PaginasWiki` (
  `ID_Pagina` VARCHAR(50) NOT NULL,
  `ID_Materia` VARCHAR(50) NOT NULL,
```

```

`ID_Archivo_Principal` VARCHAR(50) NULL DEFAULT NULL,
`Tipo_Actividad` ENUM('Tarea', 'Proyecto', 'Ensayo', 'Investigación', 'Diagrama', 'Otro') NOT NULL,
`Titulo` VARCHAR(200) NOT NULL,
`Contenido_Actual` TEXT NOT NULL,
`Esta_Bloqueada` TINYINT(1) NOT NULL DEFAULT '0',
`Votos_Positivos` INT NOT NULL DEFAULT '0',
`Votos_Negativos` INT NOT NULL DEFAULT '0',
`Fecha_Creacion` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
`Fecha_Actualizacion` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
PRIMARY KEY (`ID_Pagina`),
CONSTRAINT `fk_Pagina_Archivo` FOREIGN KEY (`ID_Archivo_Principal`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_Archivos` (`ID_Archivo`) ON DELETE SET NULL,
CONSTRAINT `fk_Pagina_Materia` FOREIGN KEY (`ID_Materia`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_Materias` (`ID_Materia`) ON DELETE CASCADE)
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;

```

```

-- Table `tbl_WikiUnach_Comentarios`
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_Comentarios` (
`ID_Comentario` VARCHAR(50) NOT NULL,
`ID_Pagina` VARCHAR(50) NOT NULL,
`ID_Usuario` VARCHAR(50) NOT NULL,
`Contenido` TEXT NOT NULL,
`ID_Archivo_Adjunto` VARCHAR(50) NULL DEFAULT NULL,
`Fecha_Creacion` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (`ID_Comentario`),
CONSTRAINT `fk_Comentario_Archivo` FOREIGN KEY (`ID_Archivo_Adjunto`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_Archivos` (`ID_Archivo`) ON DELETE SET NULL,
CONSTRAINT `fk_Comentario_Pagina` FOREIGN KEY (`ID_Pagina`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_PaginasWiki` (`ID_Pagina`) ON DELETE CASCADE,
CONSTRAINT `fk_Comentario_Usuario` FOREIGN KEY (`ID_Usuario`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_Usuarios` (`ID_Usuario`))
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;

```

```

-- Table `tbl_WikiUnach_Etiquetas`
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_Etiquetas` (
`ID_Etiqueta` VARCHAR(50) NOT NULL,
`Nombre_Etiqueta` VARCHAR(50) NOT NULL,
PRIMARY KEY (`ID_Etiqueta`),
UNIQUE INDEX `Nombre_Etiqueta_UNIQUE` (`Nombre_Etiqueta` ASC) VISIBLE)
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;

```

```

-- Table `tbl_WikiUnach_Pagina_Etiquetas`
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_Pagina_Etiquetas` (
`ID_Pagina` VARCHAR(50) NOT NULL,
`ID_Etiqueta` VARCHAR(50) NOT NULL,
PRIMARY KEY (`ID_Pagina`, `ID_Etiqueta`),

```



```

CONSTRAINT `fk_PE_Etiqueta` FOREIGN KEY (`ID_Etiqueta`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_Etiquetas` (`ID_Etiqueta`) ON DELETE CASCADE,
CONSTRAINT `fk_PE_Pagina` FOREIGN KEY (`ID_Pagina`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_PaginasWiki` (`ID_Pagina`) ON DELETE CASCADE)
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;

-- Table `tbl_WikiUnach_RevisionesWiki`
CREATE TABLE IF NOT EXISTS `WikiUnach`.`tbl_WikiUnach_RevisionesWiki` (
  `ID_Revision` VARCHAR(50) NOT NULL,
  `ID_Pagina` VARCHAR(50) NOT NULL,
  `ID_Usuario` VARCHAR(50) NOT NULL,
  `Instantanea_Contenido` TEXT NOT NULL,
  `Mensaje_Commit` VARCHAR(255) NOT NULL,
  `Fecha_Creacion` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`ID_Revision`),
  CONSTRAINT `fk_Revision_Pagina` FOREIGN KEY (`ID_Pagina`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_PaginasWiki` (`ID_Pagina`) ON DELETE CASCADE,
  CONSTRAINT `fk_Revision_Usuario` FOREIGN KEY (`ID_Usuario`)
REFERENCES `WikiUnach`.`tbl_WikiUnach_Usuarios` (`ID_Usuario`))
ENGINE = InnoDB DEFAULT CHARACTER SET = utf8mb4;

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;

```

Vistas (Implementación Programática)

Por decisiones de arquitectura y rendimiento, en el proyecto WikiUnach se optó por no declarar Vistas (Views) estáticas en el motor de MySQL. En su lugar, el concepto de "Vistas" se implementó de manera programática dentro de la capa de acceso a datos en la aplicación cliente (C#).

Esta aproximación permite inyectar parámetros de forma dinámica (MySQLCommand.Parameters), reducir el acoplamiento directo con el motor relacional y aprovechar las variables de sesión en memoria sin necesidad de cruzar tantas credenciales hacia la base de datos.

A continuación, se documentan las Vistas Programáticas más relevantes del sistema, representadas por consultas complejas (JOIN, UNION y Subconsultas) embebidas en los formularios.

Vista de Detalles de Publicación y Autoría (vw_DetallesPublicacion)

Implementada en FrmDetallesTarea.cs. Esta vista unifica los datos de una página Wiki con su materia correspondiente y utiliza una subconsulta correlacionada para buscar en el historial de revisiones (tbl_WikiUnach_RevisionesWiki) quién fue el usuario original que hizo el primer "commit" (Fecha mínima de creación), obteniendo así al "Autor" original.

```
SELECT
    pw.Titulo,
    pw.Contenido_Actual,
    pw.Tipo_Actividad,
    pw.Votos_Positivos,
    pw.Votos_Negativos,
    pw.Fecha_Creacion,
    pw.Fecha_Actualizacion,
    pw.ID_Materia,
    m.Nombre_Materia,
    u.Nombre AS NombreAutor
FROM tbl_WikiUnach_PaginasWiki pw
INNER JOIN tbl_WikiUnach_Materias m
    ON pw.ID_Materia = m.ID_Materia
LEFT JOIN (
    SELECT r1.ID_Pagina, r1.ID_Usuario
    FROM tbl_WikiUnach_RevisionesWiki r1
    INNER JOIN (
```

```

SELECT ID_Pagina, MIN(Fecha_Creacion) AS PrimeraFecha
FROM tbl_WikiUnach_RevisionesWiki
GROUP BY ID_Pagina
) r2 ON r1.ID_Pagina = r2.ID_Pagina
AND r1.Fecha_Creacion = r2.PrimeraFecha
) r ON r.ID_Pagina = pw.ID_Pagina
LEFT JOIN tbl_WikiUnach_Usuarios u
ON r.ID_Usuario = u.ID_Usuario
WHERE pw.ID_Pagina = @idPagina;

```

Vista de Tareas Administrables del Usuario (vw_TareasUsuario)

Implementada en FrmAdministrarTareas.cs. Filtra las publicaciones activas dentro de una materia, pero utilizando una cláusula EXISTS para retornar exclusivamente aquellas donde el usuario en sesión es el responsable de la publicación inicial, permitiéndole administrarlas.

```

SELECT pw.ID_Pagina, pw.Titulo, pw.Tipo_Actividad,
pw.Contenido_Actual, pw.Fecha_Creacion,
pw.Fecha_Actualizacion, pw.Esta_Bloqueada
FROM tbl_WikiUnach_PaginasWiki pw
WHERE pw.ID_Materia = @idMat
AND EXISTS (
SELECT 1 FROM tbl_WikiUnach_RevisionesWiki r
WHERE r.ID_Pagina = pw.ID_Pagina
AND r.ID_Usuario = @idUser
AND r.Mensaje_Commit = 'Publicación inicial'
)
ORDER BY pw.Fecha_Actualizacion DESC;

```

Vista Unificada de Archivos Adjuntos (vw_ArchivosCompletos)

Implementada en FrmDetallesTarea.cs. Dado que los archivos pueden estar atados como "Archivo Principal" a la página, o como "Archivos Secundarios" en los

comentarios, esta vista utiliza un UNION para unificar ambos orígenes en una sola lista deducida y presentarlos en el panel lateral.

```
SELECT a.ID_Archivo, a.Titulo, a.Descripcion, a.URL_Archivo,
       a.Tipo_Archivo, a.Tamano_Bytes, a.Fecha_Creacion
FROM tbl_WikiUnach_Archivos a
WHERE a.ID_Archivo = (
    SELECT ID_Archivo_Principal
    FROM tbl_WikiUnach_PaginasWiki
    WHERE ID_Pagina = @id1
    AND ID_Archivo_Principal IS NOT NULL)
UNION
SELECT a.ID_Archivo, a.Titulo, a.Descripcion, a.URL_Archivo,
       a.Tipo_Archivo, a.Tamano_Bytes, a.Fecha_Creacion
FROM tbl_WikiUnach_Archivos a
INNER JOIN tbl_WikiUnach_Comentarios c
    ON a.ID_Archivo = c.ID_Archivo_Adjunto
WHERE c.ID_Pagina = @id2;
```

Vista de Historial de Revisiones (vw_HistorialRevisiones)

Implementada en FrmDetallesTarea.cs. Cruza las instantáneas de contenido con el catálogo de usuarios para mostrar la bitácora completa de cambios (quién editó, cuándo y qué mensaje dejó).

```
SELECT r.ID_Revision, r.Mensaje_Commit,
       r.Instantanea_Contenido, r.Fecha_Creacion,
       u.Nombre AS NombreUsuario,
       pw.Titulo AS TituloPagina
FROM tbl_WikiUnach_RevisionesWiki r
INNER JOIN tbl_WikiUnach_Usuarios u ON u.ID_Usuario = r.ID_Usuario
INNER JOIN tbl_WikiUnach_PaginasWiki pw ON pw.ID_Pagina = r.ID_Pagina
WHERE r.ID_Pagina = @idPagina
ORDER BY r.Fecha_Creacion DESC;
```

Consultas simples, agregadas y con múltiples tablas

```

SELECT pw.Titulo, pw.Contenido_Actual, pw.Tipo_Actividad,
       pw.Votos_Positivos, pw.Votos_Negativos, pw.Fecha_Creacion,
       pw.ID_Materia, m.Nombre_Materia, u.Nombre AS NombreAutor
FROM tbl_WikiUnach_PaginasWiki pw
INNER JOIN tbl_WikiUnach_Materias m ON pw.ID_Materia = m.ID_Materia
LEFT JOIN (
  SELECT r1.ID_Pagina, r1.ID_Usuario
  FROM tbl_WikiUnach_RevisionesWiki r1
  INNER JOIN (SELECT ID_Pagina, MIN(Fecha_Creacion) AS PrimeraFecha
              FROM tbl_WikiUnach_RevisionesWiki GROUP BY ID_Pagina) r2
            ON r1.ID_Pagina = r2.ID_Pagina AND r1.Fecha_Creacion = r2.PrimeraFecha
) r ON r.ID_Pagina = pw.ID_Pagina
LEFT JOIN tbl_WikiUnach_Usuarios u ON r.ID_Usuario = u.ID_Usuario
WHERE pw.ID_Pagina = @idPagina;

```

IMPLEMENTACION AWS

Infraestructura

El proyecto hace uso de dos pilares de infraestructura administrada en la nube de Amazon Web Services, separando estratégicamente los metadatos relacionales de los archivos binarios pesados para optimizar costos y rendimiento:

AWS RDS (Relational Database Service): Actúa como el núcleo lógico del sistema. Se desplegó una instancia con el motor MySQL (wikiunach-db) encargada de procesar las consultas, mantener la integridad referencial de las facultades y materias, y gestionar la autenticación de usuarios. Al ser un servicio administrado, AWS se encarga de los parches del sistema operativo y la disponibilidad del hardware subyacente.

AWS S3 (Simple Storage Service): Actúa como el repositorio de almacenamiento de objetos (Bucket). Se utiliza para albergar de manera segura todos los archivos multimedia, PDFs y documentos generados por la comunidad estudiantil. Esta separación evita la saturación de la base de datos relacional y aprovecha la durabilidad del 99.999999999% que ofrece la infraestructura de Amazon.

Conexión remota

La arquitectura del sistema fue diseñada para que la aplicación cliente no requiera de un servidor intermedio (Middleware o API REST) en esta fase académica. La comunicación se realiza de manera directa a través de Internet utilizando el SDK nativo `MySql.Data` de C#. El Endpoint público generado por AWS RDS permite establecer un túnel TCP directo sobre el puerto 3306. Esta configuración facilita tanto la operación diaria de la aplicación compilada como la administración remota a través de clientes gráficos especializados como MySQL Workbench desde los equipos de desarrollo.

Seguridad

La exposición de una base de datos a Internet requiere capas estrictas de seguridad, las cuales se implementaron en múltiples niveles:

Firewall y Security Groups: El firewall perimetral de AWS RDS fue configurado mediante un Grupo de Seguridad que dicta reglas de entrada (Inbound Rules). Se estableció una regla apuntando a la IP 0.0.0.0/0 estrictamente sobre el puerto 3306, permitiendo el tráfico entrante necesario para que los estudiantes puedan conectarse desde cualquier red (hogar, universidad o datos móviles).

Autenticación MySQL y Cifrado en Tránsito: Para evitar interceptaciones de red (Man-in-the-Middle), se obligó a la aplicación a utilizar el algoritmo moderno de hashing `caching_sha2_password`. Asimismo, la cadena de conexión en C# integra el parámetro `SslMode=Required`, garantizando que todo el tráfico de consultas y resultados viaje cifrado.

Políticas IAM y Control S3: El bucket de S3 se configuró como privado por defecto, bloqueando el acceso público anónimo. Se creó un usuario programático IAM específico (`wikiunach-app-user`) con políticas restringidas. La aplicación C# utiliza las credenciales de este usuario para generar "URLs Pre-firmadas" temporales, garantizando que un archivo solo pueda ser descargado durante un lapso específico y únicamente si el sistema lo autoriza.

Validación Híbrida Administrativa: Las funciones críticas del sistema (como la eliminación de cuentas desde `FrmAdminUsuarios`) están protegidas a nivel de hardware. Se desarrolló una DLL nativa (`VerificadorAdmin.dll`) utilizando programación híbrida en C++ y Ensamblador x86. Esta librería valida una secuencia secreta de acceso manipulando de forma directa los

registros del procesador (ECX, AL), comparando los caracteres ASCII de la contraseña a muy bajo nivel, haciendo que el sistema sea resistente a ataques de ingeniería inversa o decompilación del código fuente en C#.

Respaldo

El respaldo de los datos operativos y la arquitectura se realiza de manera remota y programada mediante la herramienta Data Export de MySQL Workbench. Este proceso extrae un volcado (Dump) estructurado y auto-contenido en formato .sql, incluyendo tanto el esquema DDL (creación de tablas y relaciones) como los registros DML de producción. Esta estrategia asegura que, en caso de corrupción de datos o migración de cuenta en AWS, la infraestructura universitaria completa pueda ser restaurada en minutos.



WikiUnach MySQL Backup.rar

Costo estimado

Debido al enfoque académico y de escalabilidad inicial, la arquitectura se ajustó rigurosamente para mantenerse dentro de la Capa Gratuita (Free Tier) de AWS, garantizando cero costos durante la fase de despliegue y pruebas.

Servicio	Descripción en el Proyecto	Costo en Nivel Gratuito	Costo Básico de Pago Estimado (Escalabilidad)
AWS S3	Almacenamiento binario de PDFs, imágenes y tareas estudiantiles con control de versiones.	5 GB de almacenamiento estándar y 20,000 peticiones GET al mes.	~\$0.023 USD por GB adicional almacenado al mes.
AWS RDS	Motor relacional (MySQL) para persistencia de metadatos, login seguro y control de historial.	750 horas/mes (instancia db.t4g.micro o t3.micro), 20 GB de almacenamiento SSD.	~\$15 - \$20 USD/mes para instancias base de producción.
AWS EC2	<i>No aplica.</i> El proyecto es una aplicación de escritorio nativa (WinForms cliente-servidor).	750 horas de cómputo/mes.	~\$8 - \$10 USD/mes (Únicamente si a futuro se migra a un entorno web).

APLICACIÓN DE CLIENTE

Interfaz visual

Desarrollada bajo la tecnología Windows Forms (WinForms) de .NET Framework. Para dotar a la aplicación de un aspecto moderno y profesional, se implementó un diseño altamente adaptable (Responsive) que abandona las coordenadas de píxeles fijos.

Uso de Anchor y Dock: Se utilizó estratégicamente la propiedad Dock (Fill/Top) para obligar a los paneles contenedores principales a ajustarse dinámicamente a la resolución de pantalla del usuario, manteniendo los encabezados en su lugar. Por otro lado, la propiedad Anchor (None) combinada con contenedores centralizados permitió que formularios críticos, como la tarjeta de inicio de sesión (FrmAcceso) o la creación de cuentas (FrmCrearcuenta), se mantengan flotando perfectamente al centro sin importar si la ventana se maximiza o se redimensiona.

Componentes Dinámicos: En lugar de saturar un solo formulario, el sistema renderiza la información de las tareas inyectando controles de usuario dinámicos dentro de paneles de flujo (FlowLayoutPanel), logrando una navegación fluida similar a la de una aplicación web moderna.

CRUD (Create, Read, Update, Delete)

El sistema ejecuta operaciones de bases de datos de forma transaccional, abarcando el ciclo de vida completo de la información:

Create (Inserción): Procesos de alta, como la carga de material en FrmSubirTarea. Esta operación es dual: primero carga asíncronamente el archivo físico a AWS S3 mediante el objeto TransferUtility, recupera el enlace resultante, y finalmente ejecuta una sentencia INSERT en MySQL para vincular la URL del objeto con el usuario y la materia.

Read (Lectura y Consulta): Implementado mediante Vistas Programáticas complejas dentro del código C#. Un ejemplo es FrmPrincipal, que utiliza sentencias SELECT con cláusulas WHERE y ORDER BY para poblar dinámicamente controles ComboBox en cascada (Facultad → Licenciatura → Semestre), optimizando el tiempo de respuesta.

Update (Actualización): Utilizado en la gestión de perfiles (FrmCuenta) y en el versionado de páginas Wiki (FrmRevisiones). Permite a los usuarios actualizar campos como nombres o contraseñas, ejecutando sentencias

UPDATE con validaciones previas para respetar los constraints UNIQUE de correos y matrículas.

Delete (Eliminación): Un proceso de borrado seguro implementado en FrmAdministrarTareas. El sistema ejecuta un DELETE referencial en cascada dentro de MySQL RDS; una vez que el motor de base de datos confirma el borrado de los metadatos y comentarios asociados, el cliente C# extrae la llave (Key) de la URL antigua y solicita a la API de S3 la eliminación física del objeto, evitando acumular basura digital.

Conexión con AWS

Toda la lógica de conexión está centralizada para facilitar su mantenimiento. Se utilizaron métodos asíncronos (async y await de C#) en las llamadas a la base de datos (OpenAsync, ExecuteNonQueryAsync). Esto garantiza que la latencia natural de viajar por la red hacia los servidores de AWS en la región de Ohio (us-east-2) no congele el hilo principal de la interfaz gráfica, manteniendo la aplicación responsiva para el usuario final mientras se transfieren los datos.

Ejemplos de manejo de errores

Uno de los mayores retos en aplicaciones híbridas (que usan bases de datos y almacenamiento de archivos simultáneamente) es la

inconsistencia de datos ante fallos de red. Para resolver esto, operaciones como la actualización de tareas con nuevos archivos se envuelven en bloques transaccionales (MySQLTransaction).

Si ocurre una excepción durante el proceso (por ejemplo, se logró subir el archivo a S3 pero la base de datos MySQL perdió conexión y falló el INSERT), el bloque catch ejecuta un Rollback para revertir cualquier daño en las tablas, e inmediatamente invoca una rutina de limpieza para solicitar a Amazon S3 que elimine el archivo recién subido. De esta forma, el sistema asegura que no existan registros apuntando a archivos inexistentes ni archivos huérfanos sin metadatos en la base de datos.

CONCLUSION

El desarrollo de la plataforma WikiUnach representó un ejercicio de madurez técnica y un aprendizaje integral, demostrando la capacidad del equipo para consolidar conocimientos aislados de diferentes disciplinas informáticas y transformarlos en una solución tecnológica funcional para una problemática universitaria real.

Logros Alcanzados

El proyecto trascendió el desarrollo de software tradicional. Desde la perspectiva de Bases de Datos, el diseño y modelado evidenció el impacto crítico que tiene la normalización de datos (llevada rigurosamente hasta la Tercera Forma Normal) para evitar anomalías de actualización y mantener la integridad de un esquema académico complejo. Desde la perspectiva de Cloud Computing, la integración exitosa y simultánea de Amazon RDS y Amazon S3 dotó al sistema de una infraestructura de grado empresarial, obligando al equipo a dominar conceptos avanzados de redes, reglas de firewall, políticas de identidad IAM y control de acceso programático. Finalmente, la implementación de validaciones de seguridad nativas en lenguaje Ensamblador demostró un dominio profundo sobre la arquitectura a bajo nivel del procesador, uniendo de manera exitosa los

extremos del espectro computacional: desde los registros del hardware hasta el despliegue en la nube pública.

Limitaciones y Mejoras Futuras

Si bien la arquitectura actual es robusta, existen áreas de oportunidad identificadas para iteraciones futuras. Actualmente, la responsabilidad de mantener la sincronización y ejecutar las limpiezas de archivos huérfanos ante fallas de red recae enteramente en el cliente C#. A futuro, se contempla migrar esta lógica hacia arquitecturas sin servidor (Serverless), implementando triggers con funciones AWS Lambda que auditen y limpien el bucket S3 automáticamente de forma asíncrona.

A nivel de interfaz gráfica y experiencia de usuario, el sistema de visualización de documentos está limitado a las capacidades del motor WebBrowser heredado de Windows Forms. Como principal mejora funcional, se proyecta la migración hacia tecnologías modernas como WebView2 (basado en Chromium), lo cual permitirá la previsualización nativa de formatos complejos y mejorará drásticamente el rendimiento de lectura y colaboración dentro del entorno de la biblioteca digital.